

LexCalc Pro: Specs Técnicas V2

Documento de Engenharia: Testes (Qualidade/Segurança), CI/CD Pipeline e **Hardening de Software**.

Integração de Hardening: Os testes encontram as falhas, mas o hardening garante que a superfície de ataque seja minimizada por design. Este documento unifica as táticas preventivas e reativas.

1. Estratégia de Repositório (GitHub & Git Flow)

Para um software que exige extrema confiabilidade, a manutenção deve ser estritamente versionada.

- **main** : **Estritamente Protegida**. Código só entra via Pull Request (PR) aprovado com CI/CD verde.
- **develop** : Branch de integração. É onde as features são agrupadas antes de uma release.
- Regras do GitHub: *Require status checks to pass* e *Require linear history*.

2. Pipeline de CI/CD (Testes + Hardening Checks)

Arquivo: `.github/workflows/ci.yml`

```

name: LexCalc Pro CI/CD & Hardening

on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main", "develop" ]

jobs:
  security_quality_hardening:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install Dependencies
        run: |
          pip install -r requirements.txt
          pip install black flake8 bandit safety pytest pytest-django pytest-cov pip-
audit

      - name: Linting & Padrões
        run: black --check . && flake8 .

      - name: SAST & Hardening Scan (Bandit)
        run: bandit -r core/ engines/ api/ -ll -ii

      - name: Verificação de Dependências Rigorosa (pip-audit)
        run: pip-audit --desc on

      - name: Testes Funcionais & Coverage (Pytest)
        env:
          DJANGO_SETTINGS_MODULE: lexcalc_pro.settings
        run: pytest --cov=core --cov=engines --cov-fail-under=95

```

3. Especificação de Hardening do Software (Django/API)

O Hardening (endurecimento) garante que, mesmo se um teste falhar em pegar uma vulnerabilidade específica, as camadas de defesa do framework bloqueiem a exploração.

3.1. Hardening de Configuração (settings.py)

Para o deploy em produção, o Antigravity DEVE aplicar estas configurações de segurança. Nenhuma API deve ir ao ar sem elas:

```
# lexcalc_pro/settings.py (PRODUÇÃO)

DEBUG = False # MANDATÓRIO: Impede vazamento de stacktrace e variáveis de ambiente
ALLOWED_HOSTS = ['api.lexcalc.pro'] # Lista restrita, previne Host Header Injection

# --- HTTP Strict Transport Security (HSTS) ---
SECURE_HSTS_SECONDS = 31536000 # 1 ano
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# --- Redirecionamento e Cookies Seguros ---
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True

# --- Mitigação de Clickjacking e XSS ---
X_FRAME_OPTIONS = 'DENY'
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
```

3.2. Hardening de Dependências (Hash Locking)

Ataques de "Supply Chain" ocorrem quando uma biblioteca de terceiros é comprometida. O projeto deve usar `pip-tools` para gerar hashes de todas as dependências.

Comando de Spec: `pip-compile --generate-hashes requirements.in`

Isso garante que o servidor só instale bibliotecas se o checksum criptográfico for exatamente o mesmo testado localmente.

3.3. Hardening da API (DRF)

- **Rate Limiting Global:** Aplicar `DEFAULT_THROTTLE_RATES` no DRF (ex: `'anon': '10/min', 'user': '1000/day'`) para prevenir ataques de força bruta ou DDoS no motor de cálculos.
- **Rotatividade de Tokens (JWT):** Tokens de acesso devem expirar em 15 minutos. Apenas o *Refresh Token* (armazenado com `HttpOnly`) dura mais.
- **CORS Restrito:** `CORS_ALLOWED_ORIGINS` deve conter estritamente o domínio do front-end. Nunca usar `CORS_ALLOW_ALL_ORIGINS = True` em produção.

4. Testes para Validar o Hardening e Segurança

Os testes abaixo garantem que o hardening configurado não seja desfeito acidentalmente.

4.1. Teste de Configuração Segura (Hardening Check)

Arquivo: `core/tests/test_hardening.py`

```
import pytest
from django.conf import settings

def test_production_hardening_flags():
    """Garante que as flags críticas de segurança não sejam desativadas"""
    assert settings.DEBUG is False, "CRÍTICO: DEBUG mode não pode estar ativado"
    assert settings.SECURE_SSL_REDIRECT is True
    assert settings.SESSION_COOKIE_SECURE is True
    assert settings.X_FRAME_OPTIONS == 'DENY'
    assert settings.SECURE_HSTS_SECONDS >= 31536000
```

4.2. Proteção contra BOLA / IDOR (Isolamento de Tenant)

Garante que o Tenant B não acesse os cálculos do Tenant A.

Arquivo: `api/tests/test_tenant_isolation.py`

```
@pytest.mark.django_db
def test_cross_tenant_data_leakage(api_client, user_tenant_a, user_tenant_b,
    calc_tenant_a):
    api_client.force_authenticate(user=user_tenant_b)
    response = api_client.get(f'/api/civil/calculations/{calc_tenant_a.id}/')
    # DEVE retornar 404 (oculta a existência do registro) em vez de 403.
    assert response.status_code == 404, "Falha de Hardening: Vulnerabilidade IDOR ativa"
```

5. Como Acionar Isso com o Antigravity

1. Ao iniciar a construção da Fase 4 (Exposição REST), forneça as regras da **Seção 3.1 e 3.3** para o Agente e exija que o `settings.py` seja configurado para Produção (usando variáveis de ambiente via `python-decouple` para chaves secretas).
2. Exija que o agente escreva os testes da **Seção 4** antes de implementar a lógica das Views da API. (Desenvolvimento Guiado por Testes/Segurança).